

Coleções no Python: Listas, Tuplas e Dicionários

Neste artigo abordaremos mais tipos de dados na linguagem Python. Aqui veremos os tipos que trabalham como coleções de dados, como as listas, tuplas e dicionários.

Guia do artigo:

- Listas
- Tuplas
- Dicionários
- Funções para coleções

As coleções permitem armazenar múltiplos itens dentro de uma única unidade, que funciona como um container. Neste artigo veremos três dentre as coleções mais utilizadas em Python, que são as listas, tuplas e dicionários.

Listas

Lista é uma coleção de valores indexada, em que cada valor é identificado por um índice. O primeiro item na lista está no índice 0, o segundo no índice 1 e assim por diante.

Para criar uma lista com elementos deve-se usar colchetes e adicionar os itens entre eles separados por vírgula, como mostra o **Código 1**.

```
1 programadores = ['Victor', 'Juliana', 'Samuel', 'Caio', 'Luana']
2 print(type(programadores)) # type 'list'
3 print(len(programadores)) # 5
4 print(programadores[4]) # Luana
```

Código 1. Lista no Python

Na linha 1 criamos uma variável do tipo lista chamada `programadores` contendo cinco nomes como os seus itens. Como já visto antes a função `type()` (Linha 2) traz o tipo de variável e `len()` (Linha 3) o tamanho do objeto. Observe que na linha 4 imprimimos um item da lista acessando o índice 4.

Outra característica das listas no Python é que elas são mutáveis, podendo ser alteradas depois de terem sido criadas. Em outras palavras, podemos adicionar, remover e até mesmo alterar os itens de uma lista.

No **Código 2** vemos um exemplo de como alterar uma lista.

```
1 programadores = ['Victor', 'Juliana', 'Samuel', 'Caio', 'Luana']
2 print(programadores) # ['Victor', 'Juliana', 'Samuel', 'Caio', 'Luana']
3
4 programadores[1] = 'Carolina'
5 print(programadores) # ['Victor', 'Carolina', 'Samuel', 'Caio', 'Luana']
```

Código 2. Alteração em lista

Primeiro, criamos uma lista contendo algumas strings e depois imprimimos o seu valor na linha 2. Após isso, acessamos um dos elementos dela e alteramos o valor dele para “Carolina” na linha 4.

Além de alterar elementos em listas, também é possível adicionar itens nelas, pois já vêm com uma coleção de métodos predefinidos que podem ser usados para manipular os objetos que ela contém. No caso de adicionar elementos, podemos usar o método `append()`, como veremos no **Código 3**

```
1 programadores = ['Victor', 'Juliana', 'Samuel', 'Caio', 'Luana']
2 print(programadores) # ['Victor', 'Juliana', 'Samuel', 'Caio', 'Luana']
3
4 programadores.append('Renato')
5 print(programadores) # ['Victor', 'Juliana', 'Samuel', 'Caio', 'Luana', 'Renato']
```

Código 3. Adicionando item na lista

Na linha 1 criamos a lista e na linha 2 a imprimimos. Na linha 3 usamos o método `append()`, que adiciona elementos no final de uma lista. Quando imprimimos a lista na linha 4 vemos que ela exibirá o item adicionado na última posição.

Há outra forma de adicionarmos itens na lista, que é através do método `insert()`. Ele usa dois parâmetros: o primeiro para indicar a posição da lista em que o elemento será inserido e o segundo para informar o item a ser adicionado na lista. O **Código 4** mostra como isso ocorre na prática.

```
1 programadores = ['Victor', 'Juliana', 'Samuel', 'Caio', 'Luana']
2 programadores.insert(1, 'Rafael')
3
4 print(programadores) # ['Victor', 'Rafael', 'Juliana', 'Samuel', 'Caio', 'Luana']
```

Código 4. Adicionando item na lista

No código acima adicionamos na posição 1 da lista a string 'Rafael'. O resultado que será gerado pela linha 4 será o seguinte:

```
1 | ['Victor', 'Rafael', 'Juliana', 'Samuel', 'Caio', 'Luana']
```

Assim como podemos adicionar itens em nossa lista, também podemos retirá-los. E para isso temos dois métodos: `remove()` para a remoção pelo valor informado no parâmetro, e `pop()` para remoção pelo índice do elemento na lista. Vejamos como isso funciona na lista de programadores que estamos usando como exemplo:

```
1 | programadores = ['Victor', 'Juliana', 'Samuel', 'Caio', 'Luana']
2 | print(programadores) # ['Victor', 'Juliana', 'Samuel', 'Caio', 'Luana']
3 |
4 | programadores.remove('Victor')
5 | print(programadores) # ['Juliana', 'Samuel', 'Caio', 'Luana']
```

Código 5. Removendo item da lista

A remoção é feita na linha 4 do **Código 5**, onde removemos um elemento pelo seu **valor**. Nesse caso, retiramos da lista `programadores` o item que tem a string 'Victor'. Isso fará com que a linha 5 imprima os seguintes valores:

```
1 | ['Juliana', 'Samuel', 'Caio', 'Luana']
```

Também é possível remover um item pelo seu índice, como podemos ver no **Código 6**.

```
1 | programadores = ['Victor', 'Juliana', 'Samuel', 'Caio', 'Luana']
2 | print(programadores) # ['Victor', 'Juliana', 'Samuel', 'Caio', 'Luana']
3 |
4 | programadores.pop(0)
5 | print(programadores) # ['Juliana', 'Samuel', 'Caio', 'Luana']
```

Código 6. Removendo item da lista

Usamos o método `pop()` para remover o primeiro item da lista `programadores`, gerando o seguinte resultado:

```
1 | ['Juliana', 'Samuel', 'Caio', 'Luana']
```

Note que nos códigos houve uma mudança na lista `programadores`.

No caso de tentarmos remover um item de uma posição inexistente, obtemos o erro `IndexError: pop index out of range` como veremos a seguir:

```
1 | programadores = ['Victor', 'Juliana', 'Samuel', 'Caio', 'Luana']
2 |
3 | programadores.pop(5)
```

Código 7. Erro ao remover item da lista

Na linha 3 do **Código 7**, o método `pop()` foi usado para remover um item da lista pelo seu índice. Entretanto, a posição usada como parâmetro não existe, visto que a última posição da lista é 4, e foi usado o índice 5 para a remoção.

Também podemos obter um erro ao tentar remover um item com valor inexistente de uma lista, como é mostrado no **Código 8**.

```
1 | programadores = ['Victor', 'Juliana', 'Samuel', 'Caio', 'Luana']
2 |
3 | programadores.remove('Igor')
```

Código 8. Erro ao remover item da lista

Obtemos o erro `ValueError: list.remove(x): x not in list`, pois não existe na lista `programadores` um item com a string “Igor” que queremos excluir.

Outra característica das listas é que elas podem possuir diferentes tipos de elementos na sua composição. Isso quer dizer que podemos ter strings, booleanos, inteiros e outros tipos diferentes de objetos na mesma lista. O **Código 9** mostra como isso acontece na prática.

```
1 | aluno = ['Murilo', 19, 1.79] # Nome, idade e altura
2 |
3 | print(type(aluno)) # type 'list'
4 | print(aluno) # ['Murilo', 19, 1.79]
```

Código 9. Diferentes tipos de objetos na lista

Essa característica heterogênea das listas é mostrada no exemplo acima, no qual criamos uma lista com variáveis dos tipos string, inteiro e float, que representam o nome, idade e altura, respectivamente.

Tuplas

Tupla é uma estrutura de dados semelhante a lista. Porém, ela tem a característica de ser imutável, ou seja, após uma tupla ser criada, ela não pode ser alterada. Vejamos o uso desse objeto no **Código 10**.

```
1 times_rj = ('Botafogo', 'Flamengo', 'Fluminense', 'Vasco')
2
3 print(type(times_rj)) # class='tuple'
4 print(times_rj) # ('Botafogo', 'Flamengo', 'Fluminense', 'Vasco')
```

Código 10. Tupla no Python

Acima vemos o uso de uma tupla no Python delimitada por parênteses na sua sintaxe. Na linha 1, a variável `times_rj` recebe quatro objetos do tipo string. Na linha 3, imprimimos o tipo da variável, que é uma tupla. E na linha 4, imprimimos também o conteúdo de `times_rj`, que dá o seguinte resultado:

```
1 ('Botafogo', 'Flamengo', 'Fluminense', 'Vasco')
```

Assim como é feito nas listas, podemos acessar um determinado valor na tupla pelo seu índice, como no **Código 11**.

```
1 times_rj = ('Botafogo', 'Flamengo', 'Fluminense', 'Vasco')
2
3 print(times_rj[2]) # Fluminense
```

Código 11. Tupla no Python

Uma observação a ser feita no uso de uma tupla é que se ela tiver um único item, é necessário colocar uma vírgula depois dele, pois caso contrário, o objeto que vamos obter é uma string, porque o valor do item é do tipo string. No **Código 12**, vemos esse caso:

```
1 objeto_string = ('tesoura')
2 objeto_tupla = ('tesoura',)
3
4 print(type(objeto_string)) # class 'str'
5 print(type(objeto_tupla)) # class 'tuple'
```

Código 12. Tupla no Python

É possível também acessar os itens das tuplas por meio de seus índices, da mesma forma como é feito nas listas. No **Código 13** vemos como isso ocorre na prática.

```
1 vogais = ('a', 'e', 'i', 'o', 'u')
2
3 print(vogais[1]) # e
```

Código 13. Acesso de índice da tupla

O fato da tupla ser imutável faz com que os seus elementos não possam ser alterados depois dela já criada. Vamos usar a tupla `vogais` para mostrar um exemplo desse tipo. O **Código 14** exibirá o erro `TypeError: 'tuple' object does not support item assignment`.

```
1 vogais = ('a', 'e', 'i', 'o', 'u')
2
3 vogais[1] = 'E'
```

Código 14. Erro ao alterar valor da tupla

Veja que não é possível fazer alteração nas tuplas. Diferentemente do que acontece com as listas, não podemos trocar os elementos de um objeto do tipo tupla, pois se trata de uma sequência imutável.

Para fixar esse conceito, lembre-se que as tuplas e as strings são sequências imutáveis. Já as listas são sequências mutáveis. Isso está de acordo com a [documentação oficial](#) do Python.

As tuplas devem ser usadas em situações em que não haverá necessidade de adicionar, remover ou alterar elementos de um grupo de itens. Exemplos bons seriam os meses do ano, os dias da semana, as estações do ano etc. Nesses casos, não haverá mudança nesses itens (pelo menos isso é muito improvável).

Dicionários

Os dicionários representam coleções de dados que contém na sua estrutura um conjunto de `pares chave/valor`, nos quais cada chave individual tem um valor associado. Esse objeto representa a ideia de um mapa, que entendemos como uma coleção associativa desordenada. A associação nos dicionários é feita por meio de uma chave que faz referência a um valor. No **Código 15**, vemos a estrutura de um dicionário.

```
1 dados_cliente = {  
2     'Nome': 'Renan',  
3     'Endereco': 'Rua Cruzeiro do Sul',  
4     'Telefone': '982503645'  
5 }  
6  
7 print(dados_cliente['Nome']) # Renan
```

Código 15. Dicionário no Python

A estrutura de um dicionário é delimitada por chaves, entre as quais ficam o conteúdo desse objeto. Veja que é criada a variável `dados_cliente`, à qual é atribuída uma coleção de dados que, nesse caso, trata-se de um dicionário. Na linha 7 do **Código 15**, imprimimos o conteúdo que é associado ao índice “Nome”, trazendo o resultado Renan.

” Nas listas e tuplas acessamos os dados por meio dos índices. Já nos dicionários, o acesso aos dados é feito por meio da chave associada a eles.

Para adicionar elementos num dicionário basta associar uma nova chave ao objeto e dar um valor a ser associado a ela. No **Código 16** vamos colocar a informação `Idade` em `dados_cliente`.

```
1 dados_cliente = {  
2     'Nome': 'Renan',  
3     'Endereco': 'Rua Cruzeiro do Sul',  
4     'Telefone': '982503645'  
5 }  
6  
7 print(dados_cliente) # {'Nome': 'Renan', 'Endereco': 'Rua Cruzeiro do Sul',  
8     'Telefone': '982503645'}  
9  
10 dados_cliente['Idade'] = 40  
11  
12 print(dados_cliente) # {'Nome': 'Renan', 'Endereco': 'Rua Cruzeiro do Sul',  
13     'Telefone': '982503645', 'Idade': 40}
```

Código 16. Adicionando item no Dicionário

Para remover um item do dicionário, podemos usar o método `pop()`, como vemos no **Código 17**.

```
1 dados_cliente = {  
2     'Nome': 'Renan',  
3     'Endereco': 'Rua Cruzeiro do Sul',  
4     'Telefone': '982503645'  
5 }  
6  
7 print(dados_cliente) # {'Nome': 'Renan', 'Endereco': 'Rua Cruzeiro do Sul',  
8     'Telefone': '982503645'}  
9  
10 dados_cliente.pop('Telefone', None)  
11  
12 print(dados_cliente) # {'Nome': 'Renan', 'Endereco': 'Rua Cruzeiro do Sul'}
```

Código 17. Removendo item no Dicionário

Na linha 9 do **Código 17** temos o uso do método `pop()`, usado para remover o item 'Telefone' do dicionário `dados_clientes`. Temos na chamada do método o parâmetro `None`, que é passado depois da chave a ser removida. O `None` serve para que a mensagem de erro `KeyError` não apareça devido a remoção de uma chave inexistente.

Também poderíamos usar a palavra-chave `del`, que remove uma chave e o valor associado a ela no dicionário. Isso se faz por meio da passagem no parâmetro, como vemos no exemplo do **Código 18**.

```
1 dados_cliente = {  
2     'Nome': 'Renan',  
3     'Endereco': 'Rua Cruzeiro do Sul',  
4     'Telefone': '982503645'  
5 }  
6  
7 print(dados_cliente) # {'Nome': 'Renan', 'Endereco': 'Rua Cruzeiro do Sul',  
8     'Telefone': '982503645'}  
9  
10 del dados_cliente['Telefone']  
11  
12 print(dados_cliente) # {'Nome': 'Renan', 'Endereco': 'Rua Cruzeiro do Sul'}
```

Código 18. Removendo item no Dicionário

Funções para coleções

O Python conta com funções úteis quando se trabalha com coleções. Vejamos algumas delas:

`min()` e `max()`

Veremos a seguir, as funções `min()` e `max()`. No **Código 19** temos um exemplo com elas.

```
1 numeros = [15, 5, 0, 20, 10]
2 nomes = ['Caio', 'Alex', 'Renata', 'Patrícia', 'Bruno']
3
4 print(min(numeros)) # 0
5 print(max(numeros)) # 20
6 print(min(nomes)) # Alex
7 print(max(nomes)) # Renata
```

Código 19. Funções `min()` e `max()`

Nesse código temos duas listas com os nomes `numeros` e `nomes`. A primeira lista trabalha com números, então a função `min()` retorna o menor valor dela, enquanto que a função `max()` retorna o maior valor. Já a segunda lista contém strings, o que faz com que as funções trabalhem com comparações alfabéticas. Portanto, nesse exemplo o menor valor é `Alex` e o maior `Renata`.

sum()

Para trabalhar com coleções na linguagem Python, temos também a função `sum()`, que é usada para retornar a soma de todos os elementos da coleção. No **Código 20**, temos um exemplo dela na prática:

```
1 | numeros = [1, 3, 6]
2 |
3 | print(sum(numeros)) # 10
```

Código 20. Função sum()

Como vemos no código acima, `sum()` retornou a soma dos itens da lista `numeros`. Essa função não trabalha com strings, pois não é um tipo suportado por ela. Caso fossem usadas strings, a mensagem de erro `TypeError: unsupported operand type(s) for +: 'int' and 'str'` seria exibida.

len()

A função `len()` é bastante usada em Python para retornar o tamanho de um objeto. Quando usada com coleções, retorna o total de itens que a coleção possui. Vemos isso no **Código 21**.

```
1 | paises = ['Argentina', 'Brasil', 'Colômbia', 'Uruguai']
2 |
3 | print(len(paises)) # 4
```

Código 21. Função len()

Essa função é de grande utilidade, pois pode ser usada em diversas situações, como nas estruturas condicionais e em laços de repetição por exemplo.

type()

Com a função `type()` podemos obter o tipo do objeto passado no parâmetro. No

Código 22 usamos essa função em algumas coleções.

```
1 professores = ['Carla', 'Daniel', 'Ingrid', 'Roberto']
2 estacoes = ('Primavera', 'Verão', 'Outono', 'Inverno')
3 cliente = {
4     'Nome': 'Fábio Garcia',
5     'email': 'fabio_garcia_9@outlook.com'
6 }
7
8 print(type(professores)) # list
9 print(type(estacoes)) # tuple
10 print(type(cliente)) # dict
```

Código 22. Função type()

A execução do código acima nos traz uma lista, uma tupla e um dicionário, que são as coleções trabalhadas nesse artigo.

Conclusão

Neste artigo vimos como trabalhar com **coleções de dados no Python**, como as listas, tuplas e dicionários. Vimos suas características e alguns dos métodos que podemos usar para manipular esses tipos de objetos. Com eles podemos resolver uma ampla variedade de problemas em situações que precisamos estruturar os dados.

Dependendo do contexto, uma determinada coleção é mais adequada para o uso.